

Christos Mountakis 03801

Elli Zisopoulou 03880

DISTRIBUTED **SYSTEMS**

INTERPRETER

- The entire execution logic is wrapped in `step()`. The runtime calls `step()` repeatedly for each active thread. This allows the system to pause threads, handle network messages, and context-switch.
- `Step()`:
 - It catches errors and flags the thread as finished with an error message.
 - It reads the current instruction at the PC, splits it into an "opcode" (the command) and "operands" (the arguments).
 - It uses helper methods from the state to convert variables (like `$x`) or strings (like "hello") into concrete values before acting on them.

SIMPLESCRIPT ISA

The interpreter groups SimpleScript operations into a few logical categories:

- SET: Evaluates the right-hand value and stores it in the specified variable.
- ADD, SUB, MUL, DIV, MOD: Resolves two operands as integers, performs the arithmetic, and stores the result. It includes built-in protections against division or modulo by zero.
- BGT, BGE, BLT, BLE, BEQ, BRA: Handles branching. It compares two values and, if the condition is met, overrides the PC to jump directly to a predefined label.
- SLP calculates a future timestamp and tells the scheduler to ignore the thread until that time is reached.
- PRN prints resolved values to the console, prefixing them with the unique Thread ID for clarity.
- RET marks the thread as finished.

TUPLE SPACE

- Thread-Safe Storage: In `tuple_space.py`, the space is built using a standard Python list of tuples. It uses a `threading.Lock()` to prevent race conditions (distributed communication).
- When RD or IN is called, the interpreter checks the arguments. If it sees an undeclared variable (like `@x`), it treats it as a wildcard (represented as `None` in the code).
- The `match()` function checks if the concrete values align. If they do, the interpreter takes the data in the wildcard positions and successfully assigns it to the thread's variables.
- If RD or IN cannot find a matching tuple, it forces the thread to pause execution and registers it with the scheduler until a matching tuple is finally OUT'd by another thread.

DISTRIBUTED COMMUNICATION

runtime.py file builds a peer-to-peer network to achieve this transparency:

- **Dynamic Peer Discovery (UDP Multicast):** When a new node boots up, it announces itself to a shared UDP multicast group. Existing nodes reply, allowing the system to build a dynamic network of active peers.
- **Distributed Blocking (REGISTER):** If a thread executes an **IN** or **RD** instruction for a tuple that isn't available locally, it broadcasts a TCP **REGISTER** request. Connected peers store this pending request in their local registries.
- **State Cleanup (CANCEL):** Once a thread successfully resolves its tuple (either via direct read or atomic extraction), it broadcasts a **CANCEL** message to all peers, clearing the pending request from their registries.

PREVENTING DISTRIBUTED RACE CONDITIONS

RD (Read-Only)

- **Trigger:** Node executes **OUT**.
- **Action:** **NOTIFY** message includes the actual tuple
- **Result:** The blocked thread wakes up.
- **Why:** Since **RD** doesn't alter the space, multiple nodes reading the same tuple simultaneously is perfectly safe.

IN (Consume)

- **Trigger:** Node executes **OUT**.
- **Action:** **NOTIFY** acts as a wake-up ping
- **The Problem:** If a node issues a single **IN** request, multiple remote registries might find a matching tuple at the same time and remove it
- **The Solution:** The awakened thread must send a follow-up **QUERY** message.
- **Result:** The **QUERY** atomically locks and extracts the tuple, ensuring only one node successfully consumes it.

THREAD MIGRATION

The Goal: Move a running thread from one machine to another without losing its state or data.

- **How it Works:** The **MIGRATE** command pauses the target thread.
- The system packages the entire **SimpleScriptState** (Program Counter, variables, sleep status, and pending network blocks) into a JSON payload.
- The target node receives the payload, reconstructs the state, and resumes execution from the exact instruction where it left off.

HANDLING MIGRATION EDGE CASES

A. Tuple Operations (**RD / IN**)

- **The Problem:** A migrating thread might be blocked, waiting for a tuple.
- **The Fix:**
 - **Source Node:** Broadcasts a **CANCEL** message to clear stale requests across the network.
 - **Destination Node:** Re-issues a **REGISTER** request to safely resume the wait on the new machine.

B. Sleep Operation (**SLP**)

- **The Problem:** System clocks across different machines are rarely synchronized perfectly.
- **The Fix:**
 - **Source Node:** Sends only the remaining sleep time, not the absolute timestamp.
 - **Destination Node:** Adds the remaining time to its own local clock, ensuring accurate wake-ups despite clock drift. (Assuming sending delay is approximately 0)

AUTONOMOUS LOAD BALANCING

- **The Goal:** Prevent any single runtime environment from being overwhelmed by automatically distributing the workload.
- **The Background Loop:** A dedicated load-balancing thread wakes up periodically and asks all peers for their active thread count using a **COUNT** message.
- **The Heuristic:** Calculates the global average of threads per node.
 - If the local node exceeds this threshold, it calculates its "surplus."
- **Smart Prioritization:** It doesn't just pick random threads to migrate. It migrates threads in order of least disruption:
 - **Sleeping Threads** (Highest Priority)
 - **Blocked Threads** (Waiting on **RD/IN**)
 - **Actively Running Threads** (Lowest Priority)

SHUTDOWNS & STATE PRESERVATION

- **The Problem:** If a node just crashes or exits, all running threads and stored tuples on that node are permanently lost.
- **The `shutdown` Command:** Ensures zero data loss when a node leaves the network intentionally.
- **The Process:**
 - **Safety Check:** If no other peers exist, the shutdown is canceled to prevent deleting the only active environment.
 - **Evacuation:** Migrates all local threads to the least-loaded peers in the network.
 - **Tuple Preservation:** Iterates through the local tuple space and redistributes all stored tuples to peers via `REMOTE_OUT` messages.
 - **Clean Exit:** Broadcasts a `GOODBYE` message so peers instantly remove it from their routing tables and registries.

POTENTIAL DEADLOCK DETECTION

- **The Problem:** In a distributed system, threads can easily get stuck waiting for tuples that will never arrive.
- **Distributed Awareness:** Because the load balancer constantly polls peers for their state, the system naturally gains a global view of the network.
- **The Detection Logic:** During the load-balancing phase, a node checks if **every single active thread across the entire network** is currently blocked on a **RD** or **IN** operation.
- If the entire network is frozen, it triggers a **[DEADLOCK]** warning to the console, alerting the user that intervention is required.